========== Local docker image creation

Step 1: Remove .dockerignore file
Step 2: docker install (from website – install dmg –) open docker locally
docker –version
brew install kubectl
brew install minikube
kubectl version –client
minikube version
Step 3: Create .env file in root

```
MODEL_NAME=gpt-4o-mini
```

```
QDRANT_URL=https://50d072d1-ed73-4cf2-b7d8-df6d633d337f.europe-west3-0.gcp.cloud.qdrant.io:6333
QDRANT_API_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhY2Nlc3MiOiJtIiwic3ViamVjdCI6ImFwaS1rZXk6MmIyZmI2MjMtMTU1OC00ZDlmLTg2OGEtZjAwMDE1NTZmMTRkIn0.gBKCuVR2MN5sArUNH_5BoLiFX8nDBKyDPCGr_I3QGUI
```

```
# ============================
# APP CONFIG
# ============================
APP_ENV=dev
APP_NAME=RAG Platform
APP_PORT=8000

# ============================
# OPENAI / LLM
# ============================
OPENAI_API_KEY=
MODEL_NAME=

# Optional Groq Support
GROQ_API_KEY=
USE_GROQ=false

# ============================
# QDRANT
# ============================
QDRANT_URL=
QDRANT_API_KEY=
COLLECTION_NAME=rag_docs_new
```

```
# ============================
# RAG CONFIG
# ============================
TOP_K=4
CHUNK_SIZE=800
CHUNK_OVERLAP=150

# ============================
# LANGSMITH OBSERVABILITY
# ============================
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=
LANGCHAIN_PROJECT=rag-platform-prod

# ============================
# STREAMLIT
# ============================
BACKEND_URL=http://backend-service:8000

# ============================
# SECURITY
# ============================
ENABLE_AUTH=false
JWT_SECRET_KEY=change_me_in_prod

# ============================
# LOGGING
# ============================
LOG_LEVEL=INFO
```

Step 4: docker compose up --build
Step 5: docker images
========== Local kubernetes
Step 6: **minikube start –driver=docker**
(it uses docker desktop as VM backend)

Step 7: **kubectl get nodes**
(minikube is ready)

Step 8: building the images for minikube
Minikube needs access of your docker images

eval $(minikube docker-env)
It ensures that docker build goes directly inside the minikube

Step 9: Building images again inside the minikube
Remember now docker compose will not work …u will have to run separately
Docker build -f Docker/Dockerfile.backend -t rag-backend:v1 .
Docker build -f Docker/Dockerfile.frontend -t rag-frontend:v1 .

These two commands ensure that the docker images are right now inside the local kubernetes cluster

Step 10: **Ensure the yaml files are correct**
Step 11: **kubectl apply -f k8s/**
Step 12: **kubectl get pods**
kubectl get svc

If all good
Then move to deployment

========== Deploy this in google cloud run

Step 13: brew install –cask **google-cloud-sdk**
Step 14: gcloud init
(login to google cloud from cli)
Step 15: create google cloud project
gcloud projects create edureka-demo
gcloud config set project edureka-demo

Step 16 : go to your console and link the project to your google cloud billing account
Step 17 : After this :
Enable google cloud APIs
gcloud services enable container.googleapis.com
gcloud services enable run.googleapis.com
gcloud services enable artifactregistry.googleapis.com

Step 18: It creates a directory inside artifact registry where your image will be stored
gcloud artifacts repositories create rag-repo-may9 \\n --repository-format=docker \\n --location=asia-northeast3 \\n --description="RAG app repo"

Step 19:
(cp Docker/Dockerfile.backend Dockerfile)
After this step u may have to update the dockerfile last line with
CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000}"]
gcloud builds submit \\n --tag
asia-northeast3-docker.pkg.dev/**edureka-demo**/**rag-repo-may9**/**rag-backend:v1**
(cp Docker/Dockerfile.backend Dockerfile)

Step 20 :
(cp Docker/Dockerfile.frontend Dockerfile)
CMD ["sh", "-c", "streamlit run frontend/streamlit_app.py --server.port=${PORT:-8501} --server.address=0.0.0.0"]
gcloud builds submit \\n --tag
asia-northeast3-docker.pkg.dev/**edureka-demo**/**rag-repo-may9**/**rag-frontend:v1**

After Step 20 …u will have to make small changes in the deployment and services files

Backend_deployment.yaml
Here remember your path edureka-live should be replaced with <project-id> and <rag-repo> replaced with the artifact registry name
Imagepullplicy : always

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: rag-backend
spec:
 replicas: 1
 selector:
   matchLabels:
     app: rag-backend
 template:
   metadata:
     labels:
       app: rag-backend
   spec:
     containers:
     - name: backend
      image: asia-northeast3-docker.pkg.dev/edureka-live/rag-repo/rag-backend:v1
      imagePullPolicy: Always
      ports:
```

```
      - containerPort: 8000
      envFrom:
      - secretRef:
          name: rag-secret
```

Frontend-deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: rag-frontend
spec:
 replicas: 1
 selector:
   matchLabels:
     app: rag-frontend
 template:
   metadata:
     labels:
       app: rag-frontend
   spec:
     containers:
     - name: frontend
      image: asia-northeast3-docker.pkg.dev/edureka-live/rag-repo/rag-frontend:v1
      imagePullPolicy: Always
      ports:
      - containerPort: 8501
      env:
      - name: BACKEND_URL
        value: http://backend-service:8000
```

Step 21: **gcloud run deploy** rag-backend \\n --image asia-northeast3-docker.pkg.dev/edureka-demo-may9/rag-repo-may9/rag-backend:v1 \\n --platform managed \\n --region asia-northeast3 \\n --allow-unauthenticated \\n --port 8000

It will give u an **BACKEND_URL**

(if this deploy creates problem or issues – alternative command

gcloud run deploy rag-backend \\n --image asia-northeast3-docker.pkg.dev/edureka-demo-may9/rag-repo-may9/rag-backend:v1 \\n --region asia-northeast3 \\n --allow-unauthenticated \\n --set-env-vars OPENAI_API_KEY=,QDRANT_URL=,QDRANT_API_KEY=

Step 22: gcloud run **deploy rag-frontend** \\n --image asia-northeast3-docker.pkg.dev/edureka-demo-may9/rag-repo-may9/rag-frontend:v1 \\n --region asia-northeast3 \\n --allow-unauthenticated

==========